

Khulna University of Engineering & Technology

Course No: CSE 2114

Project Report on
Customized 21-bit CPU

Submitted to: Md. Badiuzzaman Shuvo Lecturer CSE, KUET & Md. Mubtashim Abrar Nihal Lecturer CSE, KUET	Submitted by: Md. Rafid Reza Roll: 2307120 CSE, KUET
---	--

Introduction

A 21-bit CPU designing was assigned to me as a single project in the previous Lab with necessary resources provided by the instructor. Using the knowledge gained from the lab, I finally implemented a 21-bit CPU with Von Neumann architecture. It can handle 15 different instructions such as: add, and, store to memory from Accumulator register, or, sub, JMP, Load to a, HLT, JZ (jump if zero flag raised), Inc and dec on accumulator, XOR, left-shift, Store to memory from D register with 1 complex instruction: Binary Multiplication which was based on Booth's Multiplication algorithm. The CPU uses 6-bit address with 8 bit data size control rom to fetch, decode and execute each instruction. The control unit raises 21 flags in total to control the flow of logic. It utilizes 6 different bit-sized registers to hold information temporarily. Some of the important registers are: PC, MAR, MBR, Accumulator, D(for multiplication result store), Status Flag register for Alu raised flags, IR.

Specifications

Main memory:

The CPU is coupled with a 7-bit address with 21-bit word size. Since the CPU was made with Von Neumann architecture, it uses the same memory for storing instructions and storing data. To handle the Instruction and data properly, the memory has to be separated in code and data segment. Mixing these segment such as storing instruction in memory segment or vice versa could lead to unexpected result. So the programmer has to be cautious while using the CPU. In data segment the whole 1 word is treated as 21-bit integer, whereas in code segment 1 word is treated as the instruction.

Instructions:

The CPU can handle 15 instructions in total which serves different purposes. Each instruction is stored in memory first. Each instruction takes 1 word size, the formatting of these instructions are:

Opcode	Unused bits	Memory address
4 bits	10 bits	7 bits

The control unit identifies these instructions in execution cycle form IR register and raises a series of microinstruction flags upon which the instructions are executed properly. Each microinstruction are executed in 1 clock cycle. In my control unit, a single Instruction is coded to complete within 4 clock pulses in execution cycle except complex instruction (MUL) which uses $2 \times 21 = 42$ (from booth's circuit) + 4 number of clock pulses = 46 pulses in total.

Instruction Table:

Here are the instructions that the computer can do along with their microinstruction in execution cycle .

Instruction	Opcode	Memory Address (7-bit) treated as don't care?	Microinstructions
AND	0000	No	1. MBR(address part) to MAR 2. Memory to MBR 3. $A = A \& \text{MBR}$ 4. None
ADD	0001	No	1. MBR(address part) to MAR 2. Memory to MBR 3. $A = A + \text{MBR}$ 4. None
STORE_A	0010	No	1. MBR(address part) to MAR 2. Accumulator to MBR 3. MBR to memory 4. None
OR	0011	No	1. MBR(address part) to MAR 2. Memory to MBR 3. $A = A \text{MBR}$ 4. None
SUB	0100	No	1. MBR(address part) to MAR 2. Memory to MBR 3. $A = A - \text{MBR}$ 4. None
JMP	0101	Yes	1. MBR(address part) to PC 2. None 3. None 4. None
Ld_to_A	0110	No	1. MBR(address part) to MAR 2. Memory to MBR 3. MBR to Accumulator 4. None

HLT	0111	Yes	1. Control Unit clock Pulse Freezes 2. None 3. None 4. None
JZ	1000	No	1. MBR(address part) to PC (if ZF = 1) 2. None 3. None 4. None
INC	1001	Yes	1. $A = A + 1$ 2. None 3. None 4. None
DEC	1010	Yes	1. $A = A - 1$ 2. None 3. None 4. None 5.
XOR	1011	No	1. MBR(address part) to MAR 2. Memory to MBR 3. $A = A \oplus \text{MBR}$ 4. None
LShift	1100	Yes	1. $A = A \ll 1$ 2. None 3. None 4. None
STORE_D	1101	No	1. MBR(address part) to MAR 2. D to MBR 3. MBR to memory 4. None
MUL	1110	No	1. MBR(address part) to MAR 2. Memory to MBR 3. Reset registers of Booth's circuit 4. $A = A * B$

Along with the Instructions Execution cycle, Each instruction is also fetched from the memory. So each instruction has a fetch cycle. The fetch cycle generally has these microinstructions:

1. **PC to MAR**
2. **Memory To MBR**
3. **Pc = PC +1**

So, Total clock pulses needed to completely execute a instruction is :

$$\begin{aligned} T &= \text{number of microinstructions in fetch cycle} + \\ &\quad \text{Number of microinstructions in execution cycle} \\ T &= 3 + 4 = 7 \text{ (max)} \end{aligned}$$

Except for multiplication, which takes around ~50 pulses.

The ALU:

The Arithmetic and Logic Unit (ALU) can perform bitwise and, bitwise or, addition and subtraction, Left shift, increment , decrement, XOR. It takes two 21-bit data as input and outputs the result and some additional information about the result known as flags. The ALU of the custom computer outputs the following flags:

Flag Name	Description
Zero	Indicates that the output is zero.
Negative	Indicates that the output is negative.
Overflow	Indicates that overflow has occurred.
Carry	Indicates that carry or borrow has been incurred during addition or subtraction.

Registers:

Here are the registers used by the CPU of the computer:

Name	Size	Description
PC (Program Counter)	7 bits	Holds the program address (of RAM) of the next instruction to be executed. (Normally the next program address unless jump occurs.)
IR (Instruction Register)	4 bits	Holds the opcode of the instruction currently being executed.
MAR (Memory Address Register)	7 bits	Holds the memory address (RAM) of the datum to access.
MBR (Memory Buffer Register)	21 bits	Temporarily holds the value fetched from memory, ALU result, Accumulator value , D register value
AC (Accumulator)	21 bits	A general-purpose register to hold data, lower Word of multiplication result, Alu result
Flag (Flag Register)	1 bits	Only Stores the Zero Flag of ALU during arithmetic operation
D (D register)	21 bits	Stores Upper Word of multiplication result

An Example Walk-through:

The program:

v2.0 raw

0c0020 060021 040030 020022 040031

Execution:

1st instruction

The Code	0c0020
Opcode	0110 (Ld_to_A)
Memory Address	00100000
What Happens	The accumulator value is first loaded from memory location 20.

2nd instruction

The Code	060021 (hex)
Opcode	0011 (OR)
Memory Address	00100001
What Happens	Value in memory location 21 is Ored with the value in Accumulator and stored in the Accumulator.

3rd instruction

The Code	040030 (hex)
Opcode	0010 (Store_A)
Memory Address	0011000
What Happens	Value in Accumulator is stored in the memory location 30

4th instruction

The Code	020022(hex)
Opcode	0001 (ADD)
Memory Address	00100010
What Happens	Values in memory location 22 is added with accumulator and stored in accumulator

5th instruction

The Code	040031 (hex)
Opcode	0010 (Store_A)
Memory Address	00110001
What Happens	Values in Accumulator is stored in memory location 31

Conclusion

This project successfully demonstrates the design and simulation of a complete custom 21-bit CPU in Logisim Evolution. The project reinforces core computer architecture concepts including datapath design, control signal generation, two's complement arithmetic, iterative hardware algorithms, and memory-mapped instruction storage. The deliberate choice of a non-standard bit width ensures that every design decision is made consciously, providing deeper insight than simply reusing standard 32-bit components.

Future enhancements could include: pipelining the data path for higher throughput, adding a cache memory system, adding division instruction , implementing floating-point arithmetic, introducing interrupt handling, or extending the ISA with additional instructions.